

¿Por qué funciona el aprendizaje profundo?

Sobreparametrización, paisaje de pérdida y generalización

Sergio Hernández, PhD
shernandez@ucm.cl



Magíster en Data Science — Universidad Católica del Maule

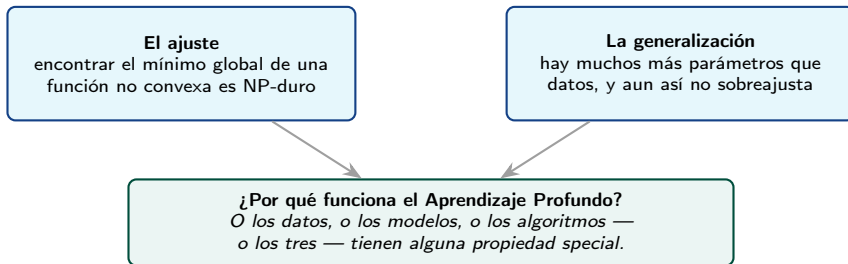
1. El caso *en contra* del aprendizaje profundo
2. Qué hace fácil el ajuste
3. Cómo es el paisaje de la función de pérdida
4. Qué determina la generalización
5. ¿Hacen falta tantos parámetros?
6. ¿Tienen que ser profundas?

Objetivo de la clase

Esta clase no presenta resultados establecidos: presenta **preguntas abiertas**. Hasta aquí vimos *cómo* se construyen y entrenan las redes; hoy preguntamos **por qué eso funciona** — y hasta dónde llega realmente nuestra comprensión.

Basado en Prince, *Understanding Deep Learning*, cap. 20.

A priori, esto no debería funcionar



- ▶ Damos por sentado que, con suficientes unidades ocultas, una red clasifica *casi cualquier* conjunto de entrenamiento a la perfección.
- ▶ Y damos por sentado que el modelo ajustado *generalizará* a datos nuevos.
- ▶ Ninguna de las dos cosas es obvia. Esta clase examina por qué ocurren.

Idea clave: Falta investigación para entender por qué el aprendizaje profundo funciona tan bien en la práctica.

Sorpresa 1: que el ajuste sea fácil

- ▶ Con MNIST-1D (40 entradas, 10 clases), una red de dos capas clasifica **perfectamente** 10.000 ejemplos con solo 43 unidades por capa (~ 4.000 parámetros).
- ▶ Pero hallar el mínimo global de una función no convexa arbitraria es **NP-duro** (Murty & Kabadi, 1987), y también lo es para ciertas pérdidas de redes neuronales (Blum & Rivest, 1992).
- ▶ El algoritmo no queda atrapado en mínimos locales ni varado en puntos silla, y además **recluta capacidad sobrante** para ajustar los datos que falten, estén donde estén.

¿Y si simplemente sobran parámetros?

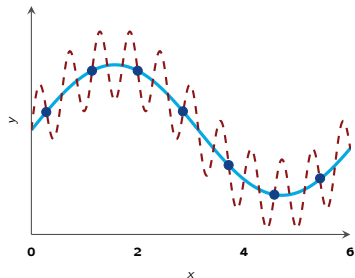
Modelo	Parám.	Datos
AlexNet	~ 60 M	~ 1 M imágenes ($\times 2048$ aumentos)
GPT-3	175 mil M	300 mil M tokens

En ninguno de los dos casos está claro que el modelo estuviera **sobreparametrizado**, y sin embargo ambos se entrenaron con éxito.

Idea clave: No es que “sobren parámetros” explique todo. Es sorprendente que podamos ajustar redes profundas de forma **fiable y eficiente**, y la explicación todavía no está cerrada.

Sorpresa 2: que generalicen

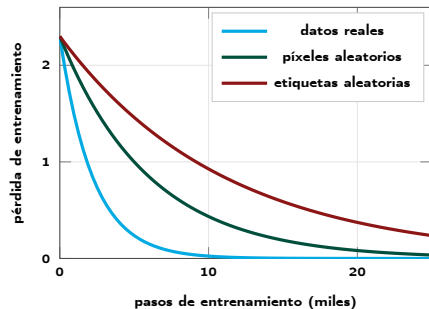
1. **Maldición de la dimensionalidad.** Si cuantizamos las 40 entradas de MNIST-1D en 10 valores, hay 10^{40} entradas posibles: 10^{36} veces más que ejemplos de entrenamiento.
2. **Funciones desmesuradas.** Una red con dos capas ocultas de ancho 400 genera hasta 10^{42} regiones lineales: unas 10^{38} regiones *por cada dato*. Casi ninguna contiene datos.
3. **Y mejora con más parámetros.** Ese modelo tiene 177.201 parámetros; si le basta uno por ejemplo, le sobran ~ 167.000 grados de libertad para hacer cualquier cosa entre los datos.



Ambas curvas pasan por los mismos datos. Con capacidad de sobra, nada obliga al modelo a elegir la azul — y aun así la elige.

Idea clave: Las regiones que *sí* contienen datos restringen al resto a comportarse razonablemente. Por qué ese contagio ocurre es, todavía, una pregunta abierta.

¿Son los datos? El experimento de Zhang et al. (2017)



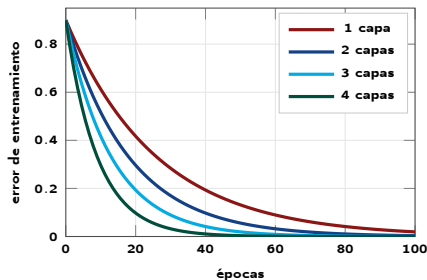
AlexNet sobre CIFAR-10 (50.000 imágenes, 10 clases) con SGD. Adaptado de Zhang et al. (2017a), fig. 20.1 de Prince.

- ▶ **Hipótesis:** ajustar es fácil porque las funciones del mundo real son “simples”.
- ▶ **Prueba:** destruyamos toda estructura. (i) Reemplazar cada imagen por ruido gaussiano. (ii) Permutar las etiquetas al azar.
- ▶ **Resultado:** la red **sigue ajustando** el conjunto, solo que más lento.

Conclusión incómoda

Las propiedades del conjunto de datos **no son** lo que hace fácil el ajuste. La red memoriza igual de bien un mapeo sin ninguna estructura.

¿Es la regularización? ¿Es el ruido de SGD?



4.000 ejemplos MNIST-1D con etiquetas aleatorias, gradiente de lote completo, inicialización He, sin momento ni regularización, $\eta = 0,0025$. Los cuatro modelos tienen ~ 15.200 parámetros. Fig. 20.2 de Prince.

Regularización

Zhang et al. mostraron que **ni L_2 (weight decay) ni Dropout** hacen falta para ajustar datos aleatorios. Queda la regularización implícita del paso finito, pero esta crece con η — y ajustar no se vuelve más fácil con η grande.

Estocasticidad

Keskar et al. (2017) ajustan bien con lotes de 5.000–6.000 imágenes, eliminando casi toda la aleatoriedad. La figura usa lote **completo**: proceso determinista, sin regularización, etiquetas sin estructura, y el error igual llega a cero.

Idea clave: Esto sugiere que estas funciones de pérdida **quizás no tengan mínimos locales** en absoluto. Ojo: no lo demuestra — ver *Problema 20.3*.

Sobrep parametrización: un pajar de agujas

- ▶ Sobreparametrizar implica una **familia enorme de soluciones degeneradas**: casi siempre existe *alguna* dirección en la que mover los parámetros reduce la pérdida.
- ▶ En la práctica las redes están sobreparametrizadas por **uno o dos órdenes de magnitud**.
- ▶ Resultados teóricos: si la red es lo bastante ancha, SGD converge al mínimo global (Du et al., 2019a,b; Allen-Zhu et al., 2019; Zou et al., 2020).
- ▶ **Pero** las hipótesis son irreales: Du et al. exigen ancho $\Omega[l^4 K^2]$ (l datos, K capas); Nguyen & Hein suponen ancho mayor que el tamaño del conjunto.

“... la degeneración de las soluciones cambia la naturaleza del problema: de buscar una aguja en un pajar, a buscar un pajar de agujas.”

— Sejnowski (2020)



una aguja



muchas agujas

Idea clave: La sobreparametrización es, casi con certeza, el factor más importante para que el ajuste sea fácil. Pero la teoría todavía **no explica** el desempeño empírico.

Función de activación (sí importa)

Si la activación solo varía en una porción pequeña del rango de entrada, la red es **más difícil de ajustar**. Sigmoides y tanh tienen colas planas: donde la activación es casi constante el gradiente es casi nulo y el mecanismo de mejora desaparece. ReLU varía en medio rango; Leaky ReLU, en todo.

Inicialización (importa poco)

Xavier/He son necesarias para evitar gradientes explosivos o evanescentes en redes profundas. Pero Liu et al. (2023c) muestran que multiplicar la varianza de He solo hace falta **más iteraciones**: no impide el ajuste. Refleja la distancia extra que deben recorrer los pesos.

Profundidad: la hipótesis del billete de lotería

Frankle & Carbin (2019): (i) entrenar la red, (ii) podar los pesos de menor magnitud, (iii) **reentrenar desde los mismos pesos iniciales**. El resultado iguala o supera al original.

Si en cambio se reinician los pesos al azar, **no funciona**.

⇒ la red sobreparametrizada contenía **subredes entrenables** suficientes por sí solas: los *billetes ganadores*.

No hay evidencia definitiva de que la pérdida sea más o menos convexa al aumentar la profundidad: los problemas de gradientes explosivos, evanescentes o *astillados* son (discutiblemente) numéricos.

Idea clave: Quizá lo que varía con la profundidad, a igual número de parámetros, es el **número efectivo de subredes** disponibles. La caracterización precisa de esa idea todavía no existe.

Balance: qué importa para ajustar y qué no

Factor	¿Importa?	Evidencia
Sobrep parametrización	Sí	Familia degenerada de soluciones; garantías de convergencia si el ancho es suficiente
Función de activación	Sí	Colas planas $\Rightarrow$ gradiente nulo; ReLU $>$ sigmoide
Estructura de los datos	No	Se ajustan píxeles y etiquetas aleatorias (Zhang et al., 2017a)
Regularización	No	Sin L_2 ni Dropout el ajuste igual llega a cero
Aleatoriedad de SGD	No	Lote completo determinista también ajusta
Inicialización	Poco	Cambia la velocidad, no el resultado (salvo gradientes inestables)
Profundidad	Poco claro	Redes profundas ajustan en menos épocas, pero puede ser un artefacto de la inicialización

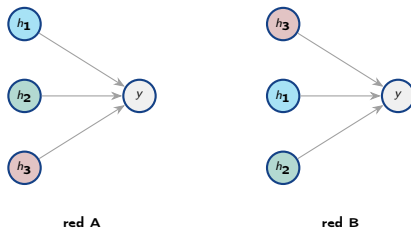
Idea clave: El resultado más incómodo del capítulo: casi todo lo que enseñamos como “buenas prácticas” de entrenamiento afecta la **generalización**, no la capacidad de ajustar. Son dos preguntas distintas.

Muchos mínimos globales (y por construcción)

Cuatro fuentes de **soluciones equivalentes** que producen exactamente la misma salida:

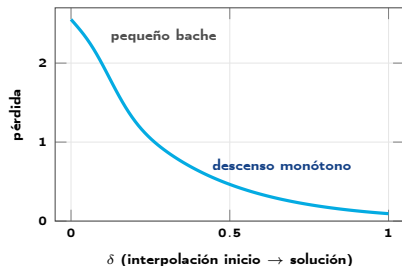
- ▶ **Permutación:** reordenar las unidades ocultas de una capa (con sus pesos) no cambia nada.
- ▶ **Canales:** en una CNN, permutar canales y núcleos de convolución.
- ▶ **Escalado de ReLU:** multiplicar el peso *antes* de un ReLU por $\alpha > 0$ y dividir el de *después* por α .
- ▶ **BatchNorm:** reinicia media y varianza de cada unidad, generando otro conjunto de redundancias.

misma función, distintos parámetros



Además, el mínimo global solo depende de la salida **en los datos de entrenamiento**. En redes sobreparametrizadas hay familias que coinciden en los datos y **difieren entre ellos**: todas son mínimos globales.

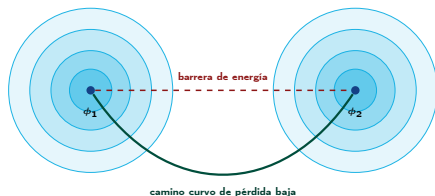
La ruta hacia el mínimo



Corte lineal entre los parámetros iniciales ($\delta=0$) y los entrenados ($\delta=1$). Goodfellow et al. (2015b); fig. 20.5a de Prince.

- ▶ En **línea recta** desde la inicialización hasta la solución la pérdida baja de forma casi monótona, en muchas arquitecturas y activaciones.
- ▶ Las trayectorias reales no son rectas, pero **viven en subespacios de baja dimensión** (Li et al., 2018b): grandes regiones casi convexas capturan la trayectoria y la encauzan.
- ▶ Li et al. (2018a): restringiendo la optimización a un subespacio **aleatorio** de 750 dimensiones (0,4% de los parámetros) se alcanza el 90% del rendimiento. Lo llaman **dimensión intrínseca**.

Idea clave: El problema de optimización tiene millones de dimensiones, pero el **camino recorrido** es de dimensión bajísima. Ahí puede estar buena parte de la explicación.



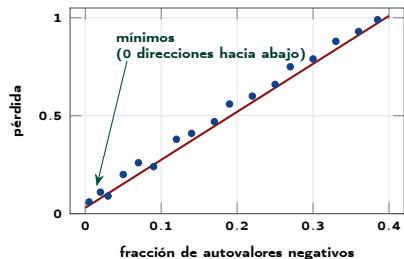
Corte de la pérdida de DenseNet sobre CIFAR-10.
Draxler et al. (2018); fig. 20.7 de Prince.

- ▶ Entre dos mínimos hallados *independientemente* la pérdida **sube** si se interpola en línea recta: no están linealmente conectados.
- ▶ Frankle et al. (2020): esa barrera **desaparece** si ambas redes se entrenan idénticamente al principio y solo después divergen. La solución queda determinada **temprano**.
- ▶ Draxler et al. (2018): siempre es posible construir un **camino curvo** donde la pérdida se mantiene baja. Cuanto más ancha y profunda la red, más fácil.

Conclusión

No hay “valles aislados”: parece haber **una sola variedad conexa** de pérdida baja.

Curvatura: puntos silla y la zona Ricitos de Oro



Dauphin et al. (2014); fig. 20.8 de Prince.

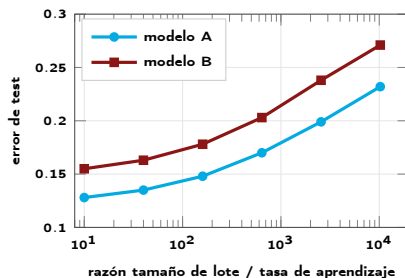
- ▶ En puntos con gradiente cero, la proporción de direcciones que **bajan** disminuye a medida que baja la pérdida.
- ▶ Es decir: los puntos críticos con pérdida alta son **puntos silla**, no mínimos. Los verdaderos mínimos aparecen todos con pérdida baja.
- ▶ Baldi & Hornik (1989) ya lo habían probado en redes superficiales: solo puntos silla, ningún mínimo local.

Zona Goldilocks

Fort & Scherlis (2019): la curvatura de la superficie es **inusualmente positiva** cuando la norma ℓ_2 de los pesos cae en cierto rango. Las inicializaciones de He y Xavier caen justo ahí.

Idea clave: No parece haber mínimos locales malos. La dificultad teórica (NP-dureza) es real *en el peor caso*, pero las pérdidas que encontramos en la práctica no son el peor caso.

El algoritmo elige a cuál mínimo llegar

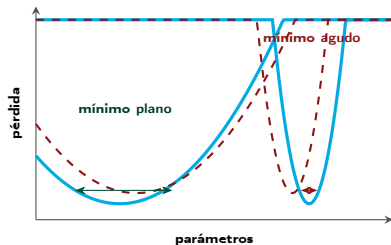


CIFAR-10. Adaptado de He et al. (2019); fig. 20.10 de Prince.

- ▶ Como la red está sobreparametrizada, es el **proceso de entrenamiento** el que decide a cuál de los mínimos degenerados se llega.
- ▶ SGD generaliza mejor que el gradiente de lote completo (LeCun et al., 2012).
- ▶ Lotes **pequeños** generalizan mejor cuando no hay otra regularización (Keskar et al., 2017); tasas de aprendizaje **grandes** también.
- ▶ Lo que importa es la **razón** tamaño de lote / tasa de aprendizaje (Jastrzębski et al., 2018; He et al., 2019).

Idea clave: Todo esto es coherente con que SGD añade una **regularización implícita** cuya magnitud depende de η : cambia la trayectoria y la lleva a una zona que generaliza bien.

Planitud del mínimo



Azul: pérdida de entrenamiento. Rojo punteado: pérdida de test, ligeramente desalineada. Keskar et al. (2017); fig. 20.11 de Prince.

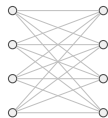
- ▶ Idea (Hochreiter & Schmidhuber, 1997): en un mínimo **plano**, un pequeño error en los parámetros — o un pequeño desajuste entre la pérdida de train y la de test — casi no cuesta.
- ▶ Longitud de descripción mínima: en mínimos anchos se necesitan **menos bits** para almacenar los pesos $\Rightarrow$ deberían generalizar mejor.
- ▶ Promediar pesos a lo largo de la trayectoria (Izmailov et al., 2018) aplanar la superficie **y** mejora la generalización.

Idea clave: Con cautela: la planitud medida puede alterarse con **reparametrizaciones triviales** de ReLU (Dinh et al., 2017), y al aleatorizar etiquetas en CIFAR el mínimo *no* se vuelve más agudo pese a que generalizar es imposible (Neyshabur et al., 2017).

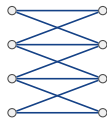
Arquitectura: el sesgo inductivo

- ▶ El **sesgo inductivo** de una red lo fija su arquitectura, y elegirla bien mejora drásticamente la generalización.
- ▶ Las **CNN** suponen que las estadísticas de la entrada son iguales en todas las posiciones, y por eso comparten parámetros a lo largo del espacio.
- ▶ Los **transformers** se ajustan a datos invariantes a permutaciones.
- ▶ Las **GNN** se ajustan a datos sobre grafos irregulares.

Emparejar la arquitectura con las propiedades de los datos generaliza mejor que una red totalmente conectada genérica.



totalmente conectada
sin supuestos

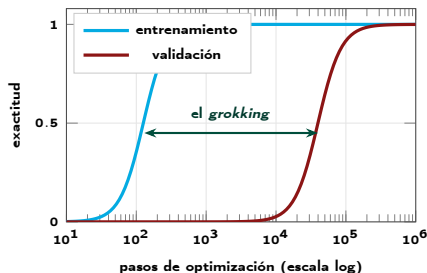


convolucional
local + compartida

Lectura

Restringir lo que la red *puede* representar no es una limitación: es información que le damos gratis sobre el problema.

La norma de los pesos y el *grokking*



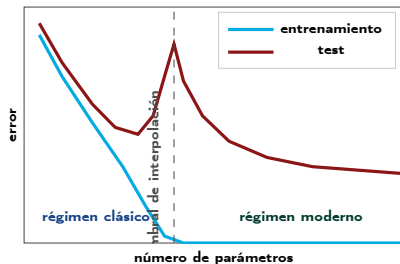
Power et al. (2022); Liu et al. (2023c); fig. 20.13 de Prince.

- ▶ La norma ℓ_2 de los pesos se relaciona (indirectamente) con la **constante de Lipschitz** del modelo.
- ▶ **Muy pequeña**: el modelo no puede cambiar lo bastante rápido para capturar la función.
- ▶ **Muy grande**: varía demasiado entre los datos y no interpola con suavidad.
- ▶ Entre medio está la **zona Goldilocks**, donde también generaliza bien (Fort & Scherlis, 2019).

Explicación del *grokking*

Si la norma inicial es demasiado grande, el modelo **ajusta** pronto pero varía mucho entre los datos. La regularización va bajando la norma; al entrar en la zona Goldilocks la generalización mejora **de golpe**, miles de épocas después.

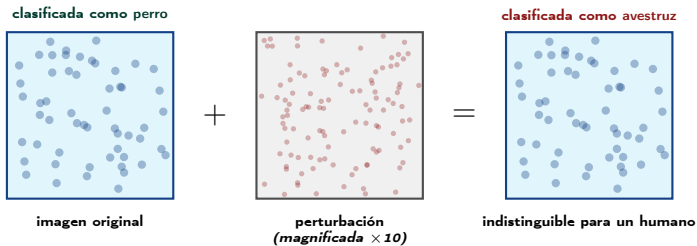
Sobreparametrización y doble descenso



- ▶ El compromiso sesgo/varianza clásico dice que pasado cierto punto el error de test debe **subir**.
- ▶ Y sube — hasta el **umbral de interpolación**, donde hay tantos parámetros como datos. Después **vuelve a bajar**.
- ▶ Explicación propuesta: sobreparametrizada, la red tiene más libertad para ser **suave** entre los datos.
- ▶ En términos de la norma: cerca del umbral el modelo se *contorsiona* para pasar por todos los puntos y su norma crece; con más anchura, la varianza de He/Glorot ($\propto 1/\text{ancho}$) hace que los pesos casi no necesiten moverse.

Idea clave: La curva en U de la teoría clásica no está mal: está **incompleta**. Es solo la mitad izquierda del dibujo.

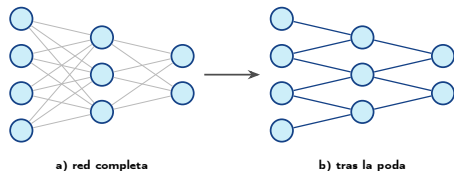
Salir de la variedad de datos: ejemplos adversarios



- ▶ Se perturba una imagen bien clasificada siguiendo el **gradiente de la salida respecto de la entrada**, hasta que la clase cambia.
- ▶ Uno esperaría algo a medio camino entre perro y avestruz. En la práctica la imagen se ve **igual**.
- ▶ Hay posiciones **cercanas pero no sobre** la variedad de datos que se clasifican mal.
- ▶ Mejor explicación actual (Ilyas et al., 2019): no es falta de robustez fuera de la distribución; el ataque **explota señales reales** de la distribución de entrenamiento, de norma pequeña e imperceptibles para nosotros.

Szegedy et al. (2014); Goodfellow et al. (2015a); fig. 20.14 de Prince. Además, D'Amour et al. (2020) muestran que modelos idénticos con distinta semilla varían enormemente sobre datos corrompidos.

Poda (*pruning*)



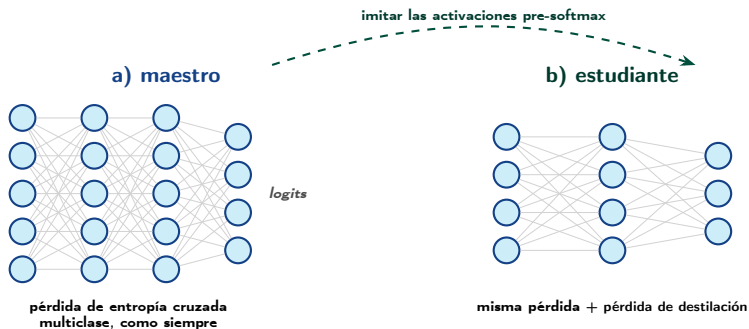
Se eliminan pesos — normalmente por **magnitud absoluta**, a veces por la segunda derivada de la pérdida — y luego se hace *fine-tuning*. También se podan unidades ocultas, canales de una CNN o capas enteras de una ResNet.

Resultados

- ▶ Han et al. (2016): VGG en ImageNet mantiene buen desempeño reteniendo el **8 %** de los pesos.
- ▶ Frankle & Carbin (2019), iterando la poda con reinicio a los pesos originales: VGG-19 (138 M parámetros) reducida un **98,5 %** en CIFAR-10; ResNet-50 (25,6 M) reducida un **80 %** en ImageNet.

Pero ojo

VGG tenía ~ 100 veces más parámetros que datos de ImageNet. Aun después de podar el 98 %, **sigue sobreparametrizada**. La poda reduce el tamaño del modelo; no demuestra que la sobreparametrización sea prescindible.



- ▶ Se entrena una red pequeña (*estudiante*) para replicar a una grande (*maestro*).
- ▶ Hinton et al. (2015): lo importante es el **patrón de información entre clases**, no solo la clase ganadora.

- ▶ Zagoruyko & Komodakis (2017) añaden transferencia de atención: aproximan una ResNet-34 (~63 M) con una ResNet-18 (~11 M) en ImageNet.
- ▶ Pero 11 M sigue siendo más que los ~1 M de ejemplos de entrenamiento.

El estado de la cuestión

- ▶ **No hay demostraciones** de rendimiento de vanguardia en conjuntos complejos con muchos menos parámetros que datos de entrenamiento.
- ▶ Ni la poda ni la destilación han cambiado ese panorama: los modelos reducidos siguen estando sobreparametrizados.
- ▶ La evidencia actual sugiere que la sobreparametrización **es necesaria** para generalizar — al menos para el tamaño y complejidad de los conjuntos que usamos hoy.

Un giro contraintuitivo

Bubeck & Sellke (2021) probaron que en D dimensiones la **interpolación suave** exige D veces más parámetros que la mera interpolación.

Su conclusión: los modelos actuales para conjuntos grandes como ImageNet **no están lo bastante sobreparametrizados**. Aumentar aún más la capacidad podría ser la clave para mejorar.

Queda abierta la pregunta de fondo: ¿es una propiedad *fundamental* de los modelos pequeños lo que les impide rendir, o simplemente **no sabemos entrenarlos**?

Idea clave: Recordemos la tensión: más parámetros hacen el ajuste *más fácil* (sección 20.2) y la generalización *mejor* (sección 20.4). Separar ambos efectos es justamente lo que aún no sabemos hacer.

La pregunta que deja abierta el teorema de aproximación universal

Recordemos

El **teorema de aproximación universal** dice que una red *superficial* puede aproximar cualquier función con precisión arbitraria, dadas suficientes unidades ocultas.

Entonces, ¿para qué la profundidad?

Evidencia a favor de la profundidad:

- ▶ Históricamente, el desempeño en ImageNet mejoró con la profundidad, hasta que entrenar se volvió difícil.
- ▶ Conexiones residuales y *batch normalization* permitieron ir más profundo, con ganancias proporcionales.
- ▶ Hoy casi todo el estado del arte (ViT, GPT-3, DALL·E-2) usa decenas o cientos de capas.

Intentos de ir más superficial

- ▶ Zagoruyko & Komodakis (2016): ResNets **más anchas y menos profundas** igualan a ResNet.
- ▶ Goyal et al. (2021): canales convolucionales **en paralelo** logran un desempeño similar con solo **12 capas**.
- ▶ Veit et al. (2016): en una ResNet lo que manda son los **caminos cortos de 5–17 capas**.

Aun así, el balance de la evidencia dice que la profundidad es crítica: incluso las redes más superficiales con buena clasificación de imágenes necesitan **más de 10 capas**.

Idea clave: Sabemos *que* la profundidad hace falta. No sabemos *por qué*.

Tres explicaciones posibles para la profundidad

1. Complejidad de la función

Las redes profundas generan muchas más regiones lineales que las superficiales con los mismos parámetros. Existen funciones “patológicas” que requieren exponencialmente más unidades en una red superficial (Eldan & Shamir, 2016; Telgarsky, 2016).

Pero: algunas de esas funciones tampoco se ajustan bien en la práctica con redes profundas (Nye & Saxe, 2018), y poco indica que las funciones reales sean patológicas.

2. Tratabilidad del entrenamiento

Quizá una red superficial *podría* rendir igual, pero es difícil encontrar una solución que ajuste y interpole bien.

Urban et al. (2017) destilaron un conjunto de 16 CNN sobre CIFAR-10 en estudiantes de distinta profundidad: las redes superficiales **no lograron** replicar al maestro, y el desempeño creció con la profundidad **a igual presupuesto de parámetros**.

3. Sesgo inductivo

Las restricciones de convoluciones y transformers no son generales: procesan localmente y **integran información gradualmente**. Eso es un buen sesgo inductivo, difícil de imponer a una red superficial.

Evidencia: la estructura de una CNN **sin entrenar** ya sirve como prior (Ulyanov et al., 2018); basta entrenar los parámetros de BatchNorm con núcleos aleatorios fijos (Frankle et al., 2021); destilar hacia arquitecturas convolucionales funciona mejor que hacia densas.

Idea clave: Las tres explicaciones son compatibles entre sí y ninguna está demostrada. Que el procesamiento local secuencial de una CNN no pueda replicarse fácilmente en pocas capas es el argumento más fuerte a favor de la profundidad.

Pregunta	Lo que sabemos
¿Por qué se ajustan?	Sobrep parametrización y función de activación son los dos factores clave
¿Importan los datos?	No: se ajustan píxeles y etiquetas aleatorias igual de bien
¿Importa la regularización?	Para ajustar, no. Para generalizar, sí
¿Cómo es el paisaje?	Sin mínimos locales aparentes; puntos silla arriba, mínimos abajo
¿Cómo se llega al mínimo?	Por un subespacio de dimensión muy baja, a una familia conexa
¿Qué mejora generalizar?	Sobrep parametrización, mínimos planos, sesgo inductivo, razón lote/ η
Doble descenso	La curva en U clásica es solo la mitad izquierda del dibujo
Norma de los pesos	Ni muy chica ni muy grande: la zona <i>Goldilocks</i> explica el <i>grokking</i>
Robustez	Ejemplos adversarios: pequeñas perturbaciones, cambios drásticos
¿Menos parámetros?	Poda y destilación ayudan, pero siguen sobrep parametrizando
¿Menos profundidad?	Parece imprescindible; no hay explicación definitiva de por qué

Lo que todavía no sabemos

- ▶ No tenemos **ninguna teoría prescriptiva** que permita predecir en qué circunstancias el entrenamiento y la generalización van a funcionar o a fallar.
- ▶ No conocemos los **límites** del aprendizaje en redes profundas.
- ▶ No sabemos si son posibles modelos **mucho más eficientes**.
- ▶ No sabemos si, dentro del mismo modelo, existen parámetros que **generalizarían mejor** que los que encontramos.

Y sin embargo

El estudio del aprendizaje profundo sigue guiado por **demostraciones empíricas**. Son innegablemente impresionantes, pero todavía no están acompañadas por una comprensión equivalente de sus mecanismos.

Idea clave: Este es un buen lugar para terminar la unidad: saber usar una herramienta y saber por qué funciona son dos cosas distintas, y en este campo la primera va muy por delante de la segunda.

- ▶ Prince, *Understanding Deep Learning*, cap. 20 (y cap. 8 sobre doble descenso)
<https://udlbook.github.io/udlbook/>
- ▶ Zhang, Bengio, Hardt, Recht & Vinyals (2017), *Understanding deep learning requires rethinking generalization*, ICLR.
- ▶ Frankle & Carbin (2019), *The lottery ticket hypothesis*, ICLR.
- ▶ Keskar, Mudigere, Nocedal, Smelyanskiy & Tang (2017), *On large-batch training for deep learning: generalization gap and sharp minima*, ICLR.
- ▶ Draxler, Veschgini, Salmhofer & Hamprecht (2018), *Essentially no barriers in neural network energy landscape*, ICML.
- ▶ Goodfellow, Shlens & Szegedy (2015), *Explaining and harnessing adversarial examples*, ICLR.
- ▶ Belkin, Hsu, Ma & Mandal (2019), *Reconciling modern machine learning practice and the bias-variance trade-off*, PNAS.

Cuadernos sugeridos (Prince, cap. 20): 20.1 datos aleatorios, 20.2 gradiente de lote completo, 20.3 billetes de lotería, 20.4 ataques adversarios.

¿Preguntas?